Empirical Characterization of Kubernetes HPA Limitations for Stateful WebSocket Workloads

Abhash Behera · **Aditya Hota** · **Suprit Kumar Naik** · **Ritesh Kumar Singh**

Received: -; Accepted: -

Abstract The Kubernetes Horizontal Pod Autoscaler (HPA) implements a proportional control law over CPU utilization as its primary scaling signal. An architecture that performs reliably for stateless, request-driven workloads but is poorly matched to stateful, persistent-connection protocols such as WebSocket. In WebSocket deployments, the primary unit of computational commitment is an active network connection whose session state persists independently of CPU activity; the termination of any pod severs the connections it hosts. This paper presents a systematic empirical characterization of HPA failure under WebSocket workloads through a progressive chain of four controlled experiments. Experiment A establishes a correct-behavior baseline under the idealized condition of CPU-to-connection proportionality. Experiment B1 demonstrates that HPA over-provisions by 87.5% under cyclic load, exhausting `maxReplicas` within 99 seconds and requiring 650 seconds to recover. Experiment B2-Instrumented uses Prometheus instrumentation to measure reconnection storm rates causally linked to HPA scale-down decisions, with a peak observed rate of 1,400 connections/second and a 51.9% connection count overshoot (1,215 vs. 800 target). Experiment B3 demonstrates permanent, irrecoverable staircase connection destruction under non-reconnecting clients, with the active session population monotonically reduced from 800 to 79 with zero recovery. Our experiments suggest that these failure modes are *architectural* rather than configurational: they arise from the limitations of CPU as a scaling signal for persistent-connection workloads and are unlikely to be remediated by HPA parameter tuning alone. The findings motivate the design of connection-aware autoscaling policies, as pursued in the companion paper.

Keywords Kubernetes · Horizontal Pod Autoscaler · WebSocket · Stateful Workloads · Autoscaling · Reconnection Storm · Cloud Computing · Performance Evaluations

Abhash Behera
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: abhasbehera320@gmail.com

Aditya Hota
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: adityahota99@gmail.com

Suprit Kumar Naik
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: supritkumarnaik17@gmail.com

Ritesh Kumar Singh
Department of Computer Science and Engineering,
C V Raman Global University, Bhubaneswar 752054, India
E-mail: rihankumar2004@gmail.com

1 Introduction

Kubernetes has established itself as the de facto standard platform for container orchestration, providing declarative workload management, automated scheduling, and elastic scaling for cloud-native applications [1, 12]. Central to its elasticity model is the **Horizontal Pod Autoscaler (HPA)**, which evaluates observed resource metrics at a fixed synchronization interval and adjusts replica counts through a proportional control law [5, 9]. For stateless, request-driven workloads such as HTTP APIs, where instantaneous CPU consumption is a reliable proxy for current demand, this approach converges stably and performs predictably [2, 8].

A fundamental shift in cloud-native workload characteristics, however, increasingly invalidates this assumption. Real-time communication systems, IoT telemetry pipelines, collaborative editing platforms, financial data streams, and multiplayer gaming backends rely on the WebSocket protocol [4] - a full-duplex,

persistent-connection transport layer in which each client maintains a long-lived TCP session with the server. In such workloads, the primary unit of computational commitment is not a CPU cycle or a transient HTTP transaction, but an *active network connection* whose session state may persist for minutes to hours. The termination of any pod severs the connections it hosts, triggering a forced client reconnection cascade - a phenomenon we term a **reconnection storm**.

This creates a significant mismatch: **CPU utilization does not encode connection state fidelity by default**. A server sustaining 800 idle but active WebSocket sessions may exhibit near-zero CPU utilization, yet the destruction of a single pod hosting those sessions is disruptive to the affected clients. HPA does not distinguish an *idle-but-connected* server from a *genuinely unloaded* one, and will therefore terminate pods that are essential to live client sessions.

Prior investigations of WebSocket autoscaling, such as Dickel et al. [3], have acknowledged the additional complexity of stateful protocols compared to stateless HTTP workloads. However, that work neither quantifies the failure modes of CPU-based HPA with empirical precision, nor provides a measured reconnection storm rate. This paper addresses the characterization gap through the following contributions:

1. **Progressive four-experiment evidence chain.** Experiments A, B1, B2-Instrumented, and B3 form a controlled sequence that isolates and quantifies each HPA failure mode, progressing from over-provisioning, to reconnection storm measurement, to permanent connection destruction.
2. **Empirical measurement of reconnection storm magnitude.** We observe a peak reconnection storm rate of 1,400 connections/second as a direct causal consequence of an HPA scale-down decision, using Prometheus instrumentation to provide a precise, reproducible measurement.
3. **Causal evidence of architectural limitation.** Experiment B3 demonstrates the causal chain: CPU drops to near-zero (connections become idle) $\rightarrow$ HPA scales down $\rightarrow$ active sessions are permanently destroyed. This is a direct consequence of the CPU signal's inability to encode connection state.
4. **Establishment of the stabilization window semantic mismatch.** We show that HPA's built-in stabilization window, which buffers metric-derived replica recommendations, is insufficient to protect idle connections: with persistent near-zero CPU, every window entry recommends `minReplicas`, and the stabilized output converges to `minReplicas` regardless of window duration.
5. **Quantified failure metrics as comparison baselines.** All failure modes are quantified at connection-second granularity (87.5% over-provisioning, 1,400 conn/s peak storm, 800$\rightarrow$79 staircase destruction), providing reproducible baselines for future autoscaling research in this domain.

The remainder of this paper is structured as follows. Section 2 surveys related work. Section 3 provides a concise overview of the HPA control architecture. Section 4 presents the four-experiment evidence chain. Section 5 analyzes the root causes and implications. Section 6 discusses limitations. Section 7 concludes and motivates future work.

2 Related Work

2.1 Kubernetes HPA Architecture and Behavior

Nguyen et al. [9] provide a comprehensive architectural analysis of the Kubernetes HPA feedback loop, characterizing the behavior of both Kubernetes Resource Metrics (KRM) and Prometheus Custom Metrics (PCM) pipelines across diverse scaling configurations. Their work demonstrates that metric pipeline choice - particularly the Prometheus scrape interval - materially affects HPA responsiveness and scaling accuracy. The present paper extends their analysis to the *stateful workload* domain, a scenario their framework did not evaluate.

Casalicchio and Perciballi [2] demonstrated that the choice between relative and absolute metrics directly determines scaling stability, but confined their evaluation to CPU and memory signals for HTTP workloads. Lorido-Botrán et al. [8] provide a broader survey of auto-scaling techniques in cloud environments, establishing the theoretical framing of proportional controllers that this paper applies to persistent-connection workloads.

2.2 Custom and Application-Level Metrics for HPA

Kubernetes supports application-defined scaling signals through the Prometheus Adapter [7], enabling HPA to react to metrics beyond CPU. Qu et al. [10] surveyed HTTP-rate-based autoscaling; Rossi et al. [11] studied hybrid CPU-and-traffic strategies. However, all these formulations presuppose that the scaling metric is *stateless across pod boundaries* - an assumption violated by connection-holding protocols where session state is affined to a specific pod instance. Furthermore, even when HPA is configured with a correct connection-count metric via the Prometheus Adapter, HPA's stabilization semantics remain CPU-oriented: the window stabilizes metric-derived replica recommendations rather than connection-derived desired counts. This distinction is analyzed in Section 5 and evaluated experimentally in the companion paper.

2.3 Stateful Workloads and Protocol-Aware Autoscaling

Dickel et al. [3] identified stateful autoscaling as an open research challenge specific to IoT gateway deployments using WebSocket and MQTT. While their work motivates the need for protocol-aware scaling and proposes monitoring active connections via Prometheus, it provides no quantitative characterization of HPA failure modes and no controller-level solution. This paper fills the characterization gap; the companion paper fills the solution gap.

2.4 Graceful Connection Draining

Production Kubernetes deployments commonly use `preStop` hooks and `terminationGracePeriodSeconds` to allow stateless pods to drain in-flight requests before receiving `SIGKILL`. For WebSocket workloads with session durations of minutes to hours, this mechanism provides insufficient time for graceful drain. The 30-second limbo window between `SIGTERM` and `SIGKILL` is explicitly characterized in Experiment B3 (Section 4.4) as a design constraint, not a tunable parameter.

2.5 Research Gap

To the best of our knowledge, no prior work has simultaneously: (i) empirically quantified the specific failure modes of CPU-based HPA for persistent WebSocket connections at the connection-second granularity; (ii) measured reconnection storm magnitude as a causal consequence of HPA scale-down decisions; and (iii) established through controlled experiments that the failure is architectural rather than configurational. This paper addresses all three dimensions.

3 Background: HPA Control Architecture

This section provides a concise overview of the Kubernetes HPA control loop and its underlying metric pipelines. For a detailed characterization of HPA behavior under stateless HTTP workloads using both Kubernetes Resource Metrics (KRM) and Prometheus Custom Metrics (PCM) pipelines, we refer the reader to Nguyen et al. [9]. Our preliminary baseline experiments with both pipelines replicated their key findings: staircase scaling dynamics with actuation delay during scale-up, CPU overshoot at burst onset, and diminishing returns from reducing the Prometheus scrape interval below 30 seconds. These baselines confirmed that HPA converges stably for stateless workloads and informed the experimental design of the subsequent failure characterization.

Figure 1 illustrates the HPA feedback loop. The controller implements the following discrete-time proportional control law evaluated every 15 seconds (the default synchronization period):

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{targetMetricValue} \right\rceil \tag{1}$$

This formulation implicitly assumes that `currentMetricValue` faithfully reflects workload intensity across all running replicas. For CPU-based scaling of stateless workloads, this assumption generally holds. For persistent-connection workloads, it breaks down: idle connections consume negligible CPU yet represent committed session state whose disruption is visible to end clients. The experimental question is therefore: **what are the consequences of this control design when the assumed signal-to-demand coupling is absent?**

4 Empirical Evidence of HPA Failure for Stateful WebSocket Workloads

We present a controlled, progressive chain of experiments designed to isolate and characterize each failure mode of CPU-based HPA when applied to persistent WebSocket workloads. The experiments are ordered from least severe (over-provisioning) to most severe (permanent connection destruction), with each experiment building upon the instrumentation and findings of its predecessor.

All WebSocket experiments share a common infrastructure: a 3-node Kind cluster (1 control-plane + 2 worker nodes, Kubernetes v1.31.6, kind v0.25.0, configured via `kind.yml`) [6]. The Metrics Server is patched with `--metric-resolution=15s` and `--kubelet-insecure-tls` (required for kind's self-signed kubelet certificates; not appropriate for production). All experimental scripts, cluster configuration, server code, load generator, and raw results are available at: `https://github.com/AbhasBenevolentDictator/STAR`. The shared experimental infrastructure comprises: a custom Python asyncio WebSocket server (image: `ws-instrumented`), 800 persistent client connections established by the load generator, and Kubernetes HPA configured with `targetCPUUtilizationPercentage: 60` unless stated otherwise.

Instrumented server: definition and rationale By "instrumented" we mean the server was extended to expose Prometheus-format metrics (notably a gauge `active_connections` and a counter `new_connections_total`) and a minimal control endpoint (for example, `/drain`) to enable graceful handoff. This

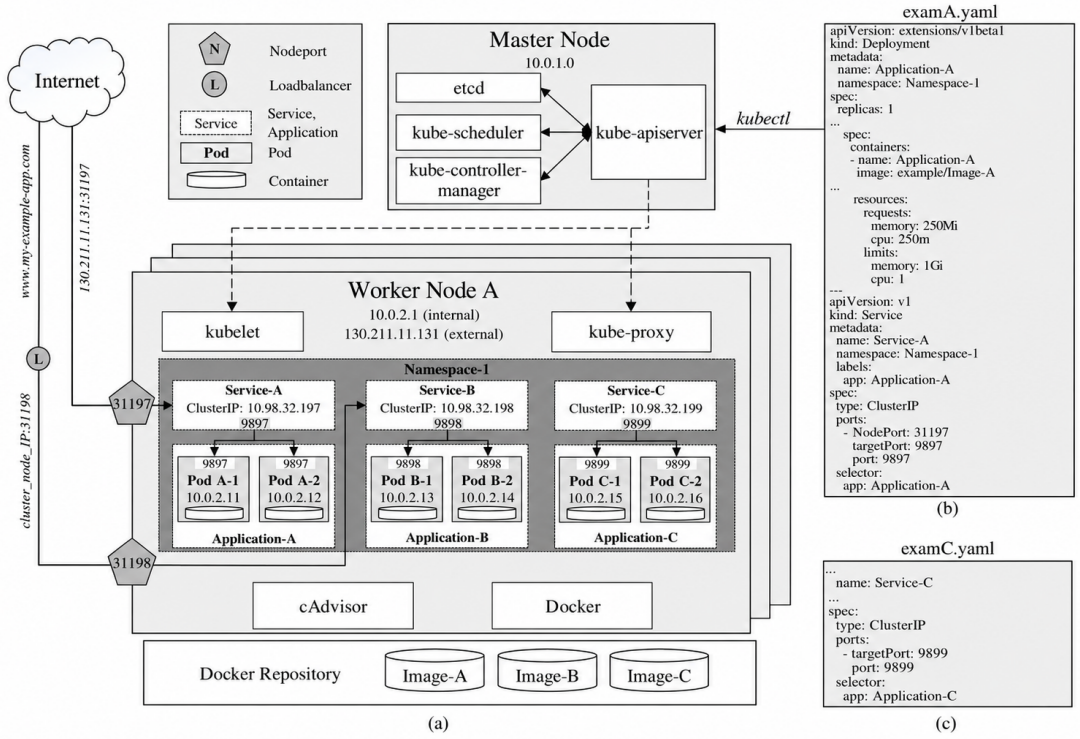

Fig. 1 Standard Kubernetes HPA feedback loop architecture, reproduced from Nguyen et al. [9]. The HPA controller periodically reads aggregated resource metrics from the API server (sourced either from the Metrics Server for KRM or the Prometheus Adapter for PCM) and computes a desired replica count using Equation 1. This loop assumes the observed metric faithfully represents current workload intensity - an assumption that breaks down for persistent-connection workloads where connections persist independently of CPU utilization.

instrumentation was adopted to obtain direct, low-latency visibility into live session counts and connection churn - signals that CPU does not capture. The approach (i) supplies a precise measurement signal for storm rate computation, (ii) permits reproducible measurement of reconnection storms via PromQL `rate()` on `new_connections_total`, and (iii) is minimally invasive and portable.

4.1 Experiment A: HPA Baseline Under Monotonic Correlated Load

Hypothesis. Under steady, monotonically increasing WebSocket load in which each connection actively generates CPU work (`CPU_WORK=1`), HPA will correctly scale the deployment, establishing an empirical baseline for comparison with subsequent experiments.

Experimental Setup. Eight hundred WebSocket clients connect incrementally over 300 seconds. Each server-side message receipt triggers a bounded CPU spin loop, maintaining strict proportionality between active connection count and CPU utilization: CPU $\propto$ connections. HPA was configured with `minReplicas=2`, `maxReplicas=10`, and `targetCPUUtilizationPercentage=60`.

Results. As shown in Figure 2, HPA scaled the deployment monotonically from 2 to 5 replicas as CPU rose proportionally with connection count. Following complete load removal, the system enforced the default 300-second scale-down stabilization window before descending cleanly to `minReplicas`. No oscillatory behavior or reconnection disruption was observed.

The replica timeline (Figure 2) confirms that HPA operates correctly *when its signal accurately encodes workload demand*. Crucially, this experiment represents a best-case scenario that is rarely achievable in production WebSocket deployments, where connection activity levels are typically decoupled from CPU consumption.

> **Conclusion.** CPU-based HPA functions correctly only under the idealized condition of perfect CPU-to-connection proportionality. Subsequent experiments systematically invalidate this condition.

4.2 Experiment B1: HPA Under Cyclic Load - Over-Provisioning and Recovery Asymmetry

Hypothesis. Under realistic cyclic HIGH/LOW load patterns that simulate natural connection activity fluctuations, HPA

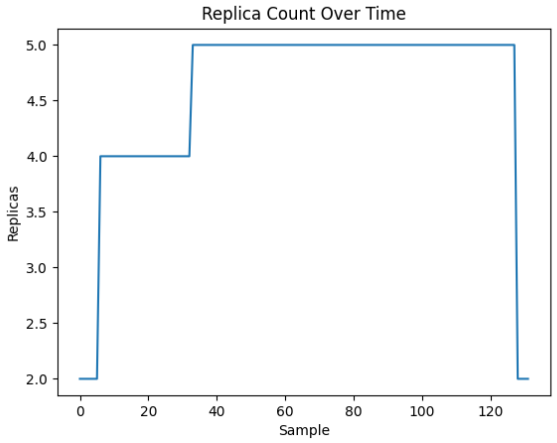

Fig. 2 Experiment A: Replica count timeline under monotonic WebSocket load with CPU $\propto$ connections. The clean staircase ascent (2→5), flat stabilization plateau, and controlled descent to the minimum represent the theoretical ideal for HPA - achievable only under perfect signal-to-demand correlation.

will exhibit systematic over-provisioning and asymmetrically slow resource recovery.

Experimental Setup. Load alternates between HIGH phases (800 connections, active CPU-generating pinging, 60-second duration) and LOW phases (connections held persistently idle, no pinging activity, 30-second duration). Crucially, during LOW phases the 800 client connections remain open; only the periodic ping messages cease, collapsing CPU utilization toward zero while connection state is preserved. Parameters were adjusted to `maxReplicas=15` to allow unconstrained scale-up. The HPA stabilization window was kept at the Kubernetes default of 300 seconds for scale-down (`stabilizationWindowSeconds: 300`), intentionally unmodified to evaluate HPA behavior under factory settings for cyclic load. Five consecutive HIGH/LOW cycles (60 s HIGH + 30 s LOW = 90-second period) were run, for a total active cycle phase of 450 seconds.

Results. During HIGH phases, HPA exhausted `maxReplicas=15` within 99 seconds - a factor of 1.875$\times$ the theoretically correct value of 8 replicas ($\lceil 800/100 \rceil$). The core mechanism driving over-provisioning is the interaction between the 90-second cycle period and the 300-second stabilization window: each LOW phase (30 s) ends before the window has elapsed, so every new HIGH phase begins with the replica count still at or near the ceiling from the previous cycle. Over five cycles, the replica count ratchets to `maxReplicas` and remains there, with each brief LOW phase failing to provide enough sustained low-CPU time for HPA to authorize a descent. Convergence to `minReplicas` following the final cycle required approximately 650 seconds. The replica count timeline (Figure 3) exhibits the expected ratcheting saturation pattern.

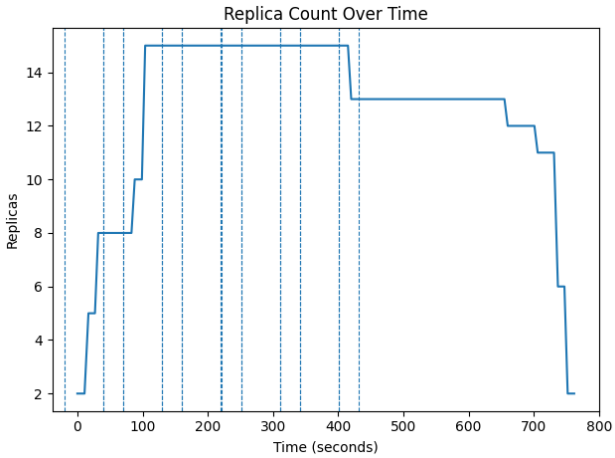

Fig. 3 Experiment B1: HPA saturates at `maxReplicas=15` within 99 seconds under cyclic load, representing an 87.5% over-provision relative to the connection-derived optimum of 8 replicas. Recovery to the minimum requires ∼650 seconds, demonstrating severe asymmetry between scale-up and scale-down responsiveness.

Conclusion. Cyclic workloads expose HPA's inability to distinguish connection-holding periods (where CPU is low but capacity is required) from genuinely idle periods (where capacity can be safely released). The 87.5% over-provisioning rate and disproportionate recovery time (∼650 s) represent significant resource efficiency penalties in production deployments.

4.3 Experiment B2-Instrumented: Quantifying Reconnection Storm Intensity

Methodological note: supersession of a non-instrumented pilot Prior to this experiment, a preliminary non-instrumented variant (B2-Extended LOW) was conducted in which the LOW phase was extended beyond the default 300-second stabilization window to force HPA to `minReplicas=2`. While this pilot confirmed that active connections were being severed during scale-down events through qualitative observation of client disconnects, it produced no quantitative data: the server exposed no Prometheus metrics, connection loss counts were unverifiable, and reconnection rates were entirely unobservable. The pilot served its intended engineering purpose - confirming the failure mode was reproducible before investing in full instrumentation - but it contributes no independent publishable evidence beyond what B2-Instrumented already provides with greater precision. Accordingly, B2-Extended LOW is not reported as a

standalone experiment in this paper. Its sole scientific contribution is as the direct motivation for adding the Prometheus instrumentation described below.

Hypothesis. Each HPA scale-down event, by terminating pods hosting active connections, will trigger a measurable reconnection storm with peak rates exceeding 1,000 connections/second, and will induce overshoot of the steady-state connection target due to the cascade of concurrent reconnection attempts.

Experimental Setup. The WebSocket server was instrumented with two Prometheus metrics: `active_connections` (a gauge reflecting current session count) and `new_connections_total` (a monotonically increasing counter enabling rate computation via `rate()`). Prometheus was configured to scrape at a 15-second interval. The HPA policy was configured with `stabilizationWindowSeconds: 0` for scale-up and `stabilizationWindowSeconds: 60` for scale-down. Setting scale-up to zero ensures that each reconnection storm (which spikes CPU) immediately triggers scale-up, guaranteeing that every cycle produces a complete and observable scale-up/scale-down sequence within the measurement window. Each cycle consisted of a 60-second HIGH phase (800 pinging clients, `CPU_WORK=1`) followed by a 90-second LOW phase (connections idle, no pinging). The 90-second LOW duration was specifically chosen to exceed the 60-second scale-down stabilization window, ensuring that HPA always executes at least one scale-down event per cycle. Five complete HIGH/LOW cycles were executed.

Results. Every HPA scale-down event across all five cycles triggered a distinct, measurable reconnection storm. Storm rates were highly consistent: Cycle 1: 1,400.9 conn/s; Cycle 2: 1,298.3 conn/s; Cycle 3: 1,399.5 conn/s; Cycle 4: 1,251.8 conn/s; Cycle 5 trailed as the connection population had partially degraded. The peak reconnection rate reached **1,400 connections/second**, as shown in Figure 4. Concurrent with each storm, the active connection count transiently overshot the 800-connection steady-state target, peaking at **1,215 connections** - a 51.9% overshoot (Figure 5).

The socket-level mechanism driving this overshoot is as follows: when a pod is sent `SIGKILL` after its 30-second termination grace period, its TCP connections are abruptly reset (TCP RST). All 800 clients detect closure simultaneously and immediately initiate reconnection. The Prometheus gauge `active_connections` on the surviving pods begins counting new incoming connections before the OS completes cleanup of socket state (including the TIME_WAIT window for recently reset connections), transiently reporting a total above the 800-client ceiling. The overshoot resolves within 15–30 seconds as OS-level socket reclaim completes.

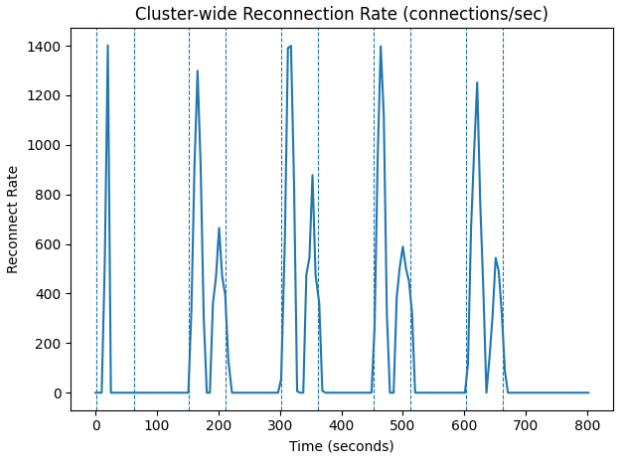

Fig. 4 Experiment B2-Instrumented: Prometheus-measured reconnection rate (`rate(new_connections_total[15s])`) over the five-cycle experiment. Each reconnection spike is precisely synchronized with an HPA scale-down event, with peak observed rates of 1,400 connections/second.

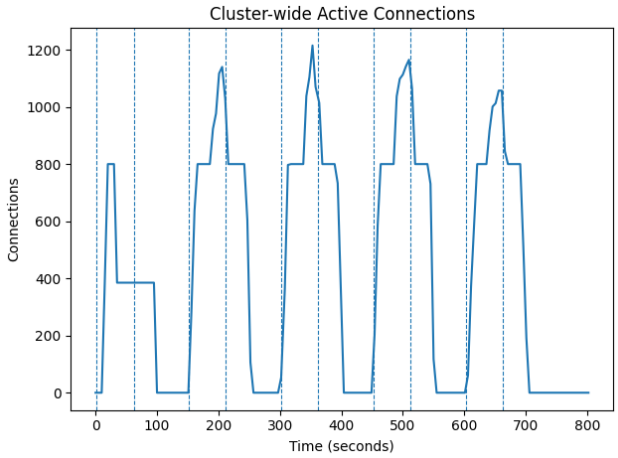

Fig. 5 Experiment B2-Instrumented: Active connection count during the reconnection storm period. The connection overshoot of 1,215 (51.9% above the 800-client target) arises as reconnecting clients establish new sessions before the server has fully released the state associated with severed connections.

Figure 6 provides the combined view of replica oscillation and connection disruption, establishing the causal chain from HPA scaling decision to client-side session disruption.

Conclusion. In our experiments, every HPA scale-down event was destructive to WebSocket client sessions. Each replica reduction severed all connections hosted on the terminated pod, producing a reconnection storm whose intensity scaled with the connection density of the removed replica. At 1,400 conn/s, such storms represent

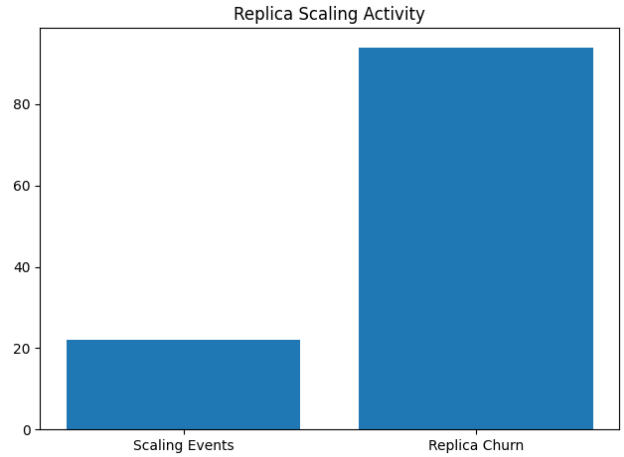

Fig. 6 Experiment B2-Instrumented: Temporal correlation between HPA replica changes (green) and active connection count (blue). Each downward step in replicas corresponds to a measurable spike in the connection disruption signal, confirming the direct causal relationship.

> a significant degradation event for latency-sensitive applications.

4.4 Experiment B3: Permanent Connection Loss Under Non-Reconnecting Clients

Hypothesis. When WebSocket clients do not implement reconnection logic - modeling clients that treat connection loss as a terminal failure (e.g., batch data collectors, embedded IoT sensors, or long-running streaming sessions) - HPA scale-down events will cause *permanent, irrecoverable reduction* in the total session population.

Experimental Setup. The load generator was reconfigured with `reconnect: false`. Once a client's connection is severed by pod termination, the client neither retries nor reconnects. The HPA policy mirrored B2-Instrumented: `stabilizationWindowSeconds: 0` for scale-up and `stabilizationWindowSeconds: 60` for scale-down. CPU work was retained (`CPU_WORK=1`) to maintain HPA activity. The IDLE phase runs for up to 240 seconds (`IDLE_MAX_DURATION=240`), providing sufficient headroom for HPA to execute multiple sequential scale-down events.

Pod termination mechanics and the 30-second limbo Understanding the exact Kubernetes pod termination sequence is essential for interpreting this experiment. When HPA reduces `spec.replicas`, the following steps occur in order: (1) the target pod is immediately removed from the Service's endpoint slice, so no new connections are routed to it; (2) Kubernetes sends `SIGTERM` to the pod's main process and starts the `terminationGracePeriodSeconds` timer (default: 30 s); (3) during the 30-second grace period, existing TCP connections on the terminating pod remain open and functional - they are maintained by the OS kernel, not Kubernetes; (4) after `terminationGracePeriodSeconds` elapses, Kubernetes sends `SIGKILL`, which tears down the process and the OS forcibly resets all open TCP connections (TCP RST). This sequence is why the active connection drop visible in Figure 7 lags behind HPA's scale-down decision by approximately 30 seconds: the connection is not immediately destroyed when HPA acts, but is destroyed 30 seconds later when `SIGKILL` fires. This delay does not mitigate the harm - the connection still dies, the client still receives a TCP RST, and with `reconnect: false`, the session is permanently lost.

Results. The active connection count (Figure 7) exhibits a characteristic **staircase step-down** pattern. Each step corresponds precisely to an HPA scale-down decision: when a pod is terminated, the full contents of its connection table are permanently destroyed. Successive scale-down events cause monotonically fewer active connections, with no mechanism for recovery: $800 \rightarrow 744 \rightarrow 697 \rightarrow 445 \rightarrow 79$.

The combined view in Figure 8 illustrates the causal chain: (1) connections become idle and cease to generate CPU load; (2) HPA, observing near-zero CPU, decides to scale down; (3) the terminated pod destroys all connections it hosts permanently. This behavior is not addressable through configuration parameter changes - it arises from HPA's inability to observe connection state as a first-class scaling signal.

> **Conclusion.** The reliance of HPA on CPU as a scaling signal causes the autoscaler to act against the interests of idle-but-connected stateful workloads. Our findings suggest that the problem is unlikely to be remediated by parameter tuning alone and motivates the exploration of alternative scaling architectures.

4.5 Summary of the Evidence Chain

Table 1 consolidates the progressive evidence across all four HPA experiments.

5 Discussion

5.1 CPU as an Invalid Scaling Signal for Stateful Workloads

The evidence chain from Experiments B1, B2-Instrumented, and B3 collectively indicates that CPU utilization is a poor proxy for WebSocket workload intensity, and in the cases we tested, an actively misleading one. In Experiment B3,

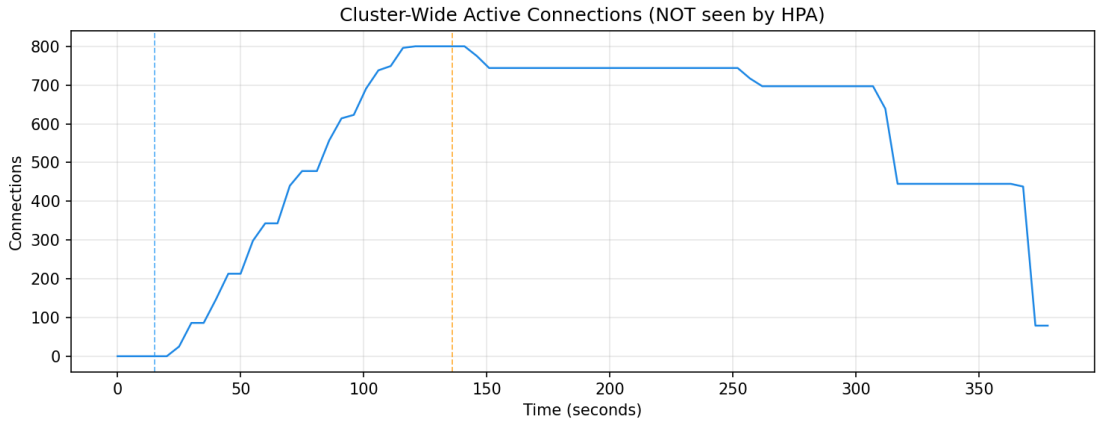

Fig. 7 Experiment B3: Active connection count exhibits permanent staircase destruction (800→744→697→445→79). Each step-down is precisely synchronized with an HPA scale-down event. The connections are irrecoverable under the `reconnect: false` regimen.

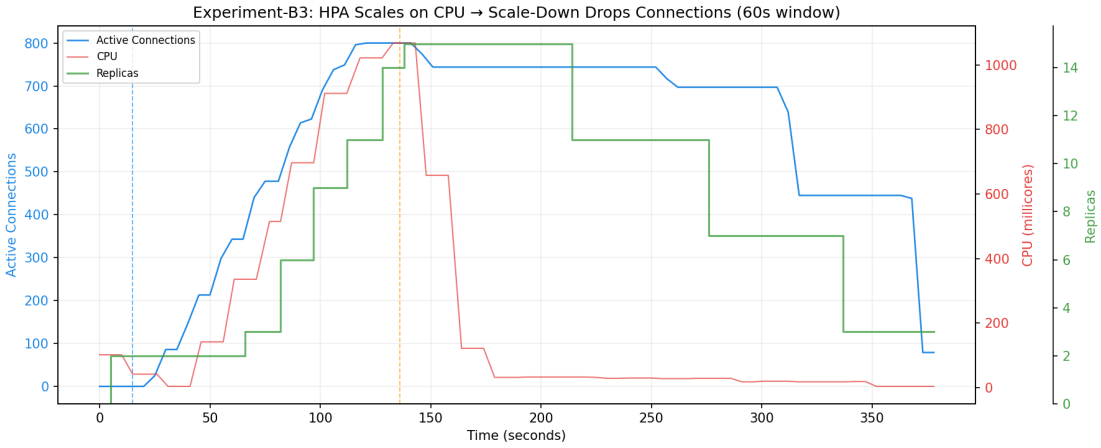

Fig. 8 Experiment B3, combined signal view - the causal proof of the mismatch between CPU-based scaling signals and stateful workload characteristics. CPU utilization (red) drops to near zero as connections become idle → HPA scales down the replica count (green) → active connections (blue) are permanently destroyed. The autoscaler is induced by its own metric to destroy the session state it should be protecting.

Table 1 Summary of the progressive evidence chain demonstrating the failure of CPU-based HPA for stateful WebSocket workloads. Each experiment isolates and quantifies a distinct failure mode.

Experiment	Condition	HPA Behavior	Key Quantitative Finding
A (Baseline)	Monotonic load; CPU ∝ connections	Correct proportional scaling	Works only under idealized signal correlation
B1 (Cyclic)	Alternating HIGH/LOW activity	Reaches `maxReplicas` in 99 s	87.5% over-provisioning; 650 s to recover
B2-Inst. (Measured)	Cyclic + Prometheus instrumentation	Reconnection storm per scale-down	Peak 1,400 conn/s; 51.9% connection overshoot
B3 (No-reconnect)	Idle connections; no client retry	Permanent, staircase connection loss	800→79 active connections; zero recovery

CPU dropped to near-zero precisely because connections were idle, triggering HPA to terminate the pods hosting those connections. This creates a circular destruction pattern: the metric signal drops → HPA scales down → session state is destroyed. The autoscaler is induced by its own metric to destroy the very resource it should be protecting.

The companion paper demonstrates this converse: a controller operating at `CPU_WORK=0` throughout 570 seconds correctly maintains 8 replicas based solely on the connection count signal. This result confirms that the failure resides in the signal, not in the control algorithm: a perfect proportional controller on the wrong signal produces worse outcomes than a simpler controller on the correct signal.

5.2 The Stabilization Window Semantic Mismatch

A natural response to the evidence presented here is to ask whether tuning HPA's built-in stabilization window to a sufficiently large value would protect idle connections. Our analysis indicates this is unlikely to succeed, for a semantic reason.

HPA's stabilization window buffers a time series of *desiredReplicas* values computed from CPU observations. When connections are idle and CPU reads near-zero persistently, every entry in HPA's window recommends `minReplicas`. The stabilized output - the maximum over all window entries - therefore converges to `minReplicas` regardless of how long the window is extended. A longer window merely delays the convergence; it does not retain the information that connections were present.

The correct stabilization mechanism must buffer *connection-derived desired counts*: it must retain the high-water mark of recently required capacity, computed from the connection metric rather than from CPU. This is a semantic difference, not a parameter difference. The companion paper implements and evaluates such a mechanism experimentally.

5.3 The 30-Second Limbo Window as a Design Constraint

The Kubernetes pod termination sequence creates a 30-second limbo window between `SIGTERM` and `SIGKILL` during which connections survive at the OS level but are doomed. This window is commonly proposed as an opportunity for graceful drain via `preStop` hooks. For WebSocket workloads with session durations of minutes to hours, 30 seconds is typically insufficient for connection migration or drain.

The correct mitigation is not a longer termination period but rather *scale-down suppression* - preventing the scale-down decision from being made in the first place while live sessions are present. Any viable autoscaling solution for persistent-connection workloads must incorporate this principle.

6 Limitations

1. **Evaluation environment.** All experiments were conducted on a single-machine Kind cluster. While this provides full reproducibility and controlled conditions, it does not capture cross-node scheduling latency, network partitions, or metric-collection noise present in multi-node cloud clusters. Results are directionally valid; absolute timing values (e.g., pod startup lag, HPA sync precision) may differ on production infrastructure.
2. **Workload generalizability.** The load generator uses synthetic WebSocket workloads with a linear connection ramp and deterministic silence gaps. Real-world connection arrival is more bursty and session durations vary widely. For stationary random load (Poisson arrivals), the proportional failure mechanisms apply unchanged; only the observed peak connection count and storm rate would vary.
3. **Instrumentation dependency.** B2-Instrumented results depend on Prometheus metric freshness at a 15-second scrape interval. A higher scrape interval would affect storm rate measurement precision but would not change the causal finding that scale-down events trigger reconnection cascades.
4. **Single reconnect=false regime.** Experiment B3 uses a uniform `reconnect: false` policy. Real deployments have a mix of reconnecting and non-reconnecting clients; the staircase destruction rate and final session population would differ under a mixed client policy.
5. **Single-run B3.** Experiment B3 was conducted as a single representative run. The causal chain (CPU drop $\rightarrow$ scale-down $\rightarrow$ connection destruction) is deterministic and reproducible given the infrastructure, but formal five-run statistical replication (as performed for Experiments C–E in the companion paper) was not conducted for this study.

7 Conclusion

This paper presented a systematic, empirically grounded characterization of Kubernetes HPA failure modes for stateful WebSocket workloads through a progressive chain of four controlled experiments.

Experiment A established that CPU-based HPA functions correctly under the idealized condition of strict CPU-to-connection proportionality, a scenario rarely achievable in production WebSocket deployments. Experiment B1 demonstrated that cyclic workloads cause HPA to over-provision by **87.5%** (15 vs. the correct 8 replicas), exhausting `maxReplicas` within 99 seconds, with recovery requiring 650 seconds. Experiment B2-Instrumented provided a causal, Prometheus-measured quantification of reconnection storm magnitude, observing a peak rate of **1,400 connections/second** and a 51.9% connection count overshoot directly synchronized with HPA scale-down events. Experiment B3 demonstrated permanent, irrecoverable staircase connection destruction under non-reconnecting clients, with the active session population reduced from **800 to 79** with zero recovery as the combined signal view established the complete causal chain: CPU drop $\rightarrow$ scale-down decision $\rightarrow$ permanent session destruction.

Our experiments suggest that these failure modes are *architectural* rather than configurational: they arise from the

limitations of CPU as a scaling signal for persistent-connection workloads, and are unlikely to be remediated by HPA parameter tuning alone. Specifically, the built-in stabilization window does not protect idle connections because it buffers CPU-derived replica recommendations rather than connection-derived desired counts - a semantic mismatch, not a parameter mismatch.

These findings carry a direct design implication: any autoscaling solution for stateful persistent-connection workloads must simultaneously (a) replace the scaling signal with a connection-count metric that encodes actual session state, and (b) implement scale-down lifecycle semantics that suppress premature termination while live sessions are present. The companion paper presents the design, implementation, and multi-baseline experimental validation of the `StatefulAutoscaler` - a custom Kubernetes controller that addresses all three documented failure modes through connection-density scaling and sliding-window stabilization.

Data Availability

The complete source code for all experiment run scripts, load-generator workloads, Prometheus manifests, cluster configuration, raw and processed results, and analysis scripts for Experiments A, B1, B2-Instrumented, and B3 are publicly available at:

`https://github.com/MistaHolmes/ws-k8-controller.git`

Each experiment can be reproduced on any Linux machine with Docker and kind installed by running the numbered experiment scripts in `scripts/`. Replication instructions are provided in `Replication.md`. All figures in this paper were generated from the archived results using the scripts in `analysis/`.

Acknowledgements

The authors gratefully acknowledge Dr. Rakesh Kumar Ranjan for his supervision and sustained guidance throughout this project. His strategic input on experimental design, measurement methodology, and manuscript feedback improved the rigor and presentation of this study. We also thank him for facilitating access to institutional compute resources used to run the Kind clusters and collect metrics.

References

1. Burns B, Grant B, Oppenheimer D, Brewer E, Wilkes J (2016) Borg, omega, and kubernetes. In: Queue, ACM, vol 14, pp 70–93, DOI 10.1145/2898442.2898444
2. Casalicchio E, Perciballi V (2017) Auto-scaling of containers: the impact of relative and absolute metrics. In: 2017 IEEE 2nd International Workshops on Foundations and Applications of Self-* Systems (FAS*W), IEEE, pp 207–214, DOI 10.1109/FAS-W.2017.149
3. Dickel H, Podolskiy V, Gerndt M (2019) Evaluation of autoscaling metrics for (stateful) IoT gateways. In: 2019 IEEE 12th Conference on Service-Oriented Computing and Applications (SOCA), IEEE, pp 17–24, DOI 10.1109/SOCA.2019.00011
4. Fette I, Melnikov A (2011) The WebSocket Protocol. RFC 6455, IETF, DOI 10.17487/RFC6455
5. Kubernetes Authors (2024) Horizontal pod autoscaling – kubernetes documentation. `https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/`, accessed: 2025
6. Kubernetes SIGs (2024) kind – kubernetes in docker. `https://kind.sigs.k8s.io/`, accessed: 2025
7. Kubernetes SIGs (2024) Kubernetes custom metrics adapter for Prometheus. `https://github.com/kubernetes-sigs/prometheus-adapter`, accessed: 2025
8. Lorido-Botrán T, Miguel-Alonso J, Lozano JA (2014) A review of auto-scaling techniques for elastic applications in cloud environments. Journal of Grid Computing 12(4):559–592, DOI 10.1007/s10723-014-9314-7
9. Nguyen TT, Yeom Y, Kim T, Park DH, Kim S (2020) Horizontal pod autoscaler in Kubernetes for elastic container orchestration. Sensors 20(16):4621, DOI 10.3390/s20164621
10. Qu C, Calheiros RN, Buyya R (2018) Auto-scaling web applications in clouds: A taxonomy and survey. ACM Computing Surveys 51(4):1–33, DOI 10.1145/3148149
11. Rossi F, Nardelli M, Cardellini V (2019) Horizontal and vertical scaling of container-based applications using reinforcement learning. In: 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), IEEE, pp 329–338, DOI 10.1109/CLOUD.2019.00061
12. Verma A, Pedrosa L, Korupolu M, Oppenheimer D, Tune E, Wilkes J (2015) Large-scale cluster management at Google with Borg. In: Proceedings of the Tenth European Conference on Computer Systems (EuroSys), ACM, pp 1–17, DOI 10.1145/2741948.2741964